

Contents

Agentic systems — working notes	1
1. The framework	1
2. Control flow — who decides the next step	1
3. Tracking and feedback	3
4. The guiding agent today	3
5. Agent registry — what goes in, what must come out	5
6. Tool catalogue — the 22 tools the guide agent has	7
7. Tasks, and whether one loop fits them	8
8. Debug sheet	11
9. Issue and fix register	12
10. One turn, end to end — the implementation walkthrough	15

Agentic systems — working notes

Running log of our agent discussions — kept to what we actually discussed. Last updated: 2026-09-17.

1. The framework

An agent is defined by its tasks, not by its model. Tasks go in, an expected outcome comes out, and everything in between is capability we chose to give it.

- **Tasks in:** each thing the agent must do, written down.
- **Expected outcome out:** what counts as done, per task. If it isn't written, the agent cannot be said to have failed.
- **The agent is non-deterministic:** same task in, a different path each run. Its shape is either one agentic loop (the model picks the next tool) or a workflow of nodes (code picks the next step).
- **Capabilities:** all it can actually do — the model, the tools, the environment. For every task, analyse which tools it needs; a task with no tool behind it is where the agent fails first.
- **Track every request:** input, path taken, outcome, cost.
- **User feedback:** when an answer is hallucinated, wrongly refused, or otherwise unusable, capture it against that request.
- **Tagged benchmark:** every failure becomes a case — the failing request, a tag, and what the right answer was. This is our own benchmark, built from real traffic.
- **Root cause, per bad case:** missing tool, wrong model, prompt gap, bad context. The tag says what went wrong; the root cause says why.
- **Fix, then replay:** the whole benchmark runs before any change ships, so improvements are measured rather than assumed.

2. Control flow — who decides the next step

Two ways to build: **workflow** (our code decides every next step — deterministic) and **dynamic** (the model's own output decides the next step). A graph can express either; the axis is who owns the routing, not whether a graph is used.

Settled rules:

- The model decides *what* to do; our code decides what it is *allowed* to do and what happens *after* it does.

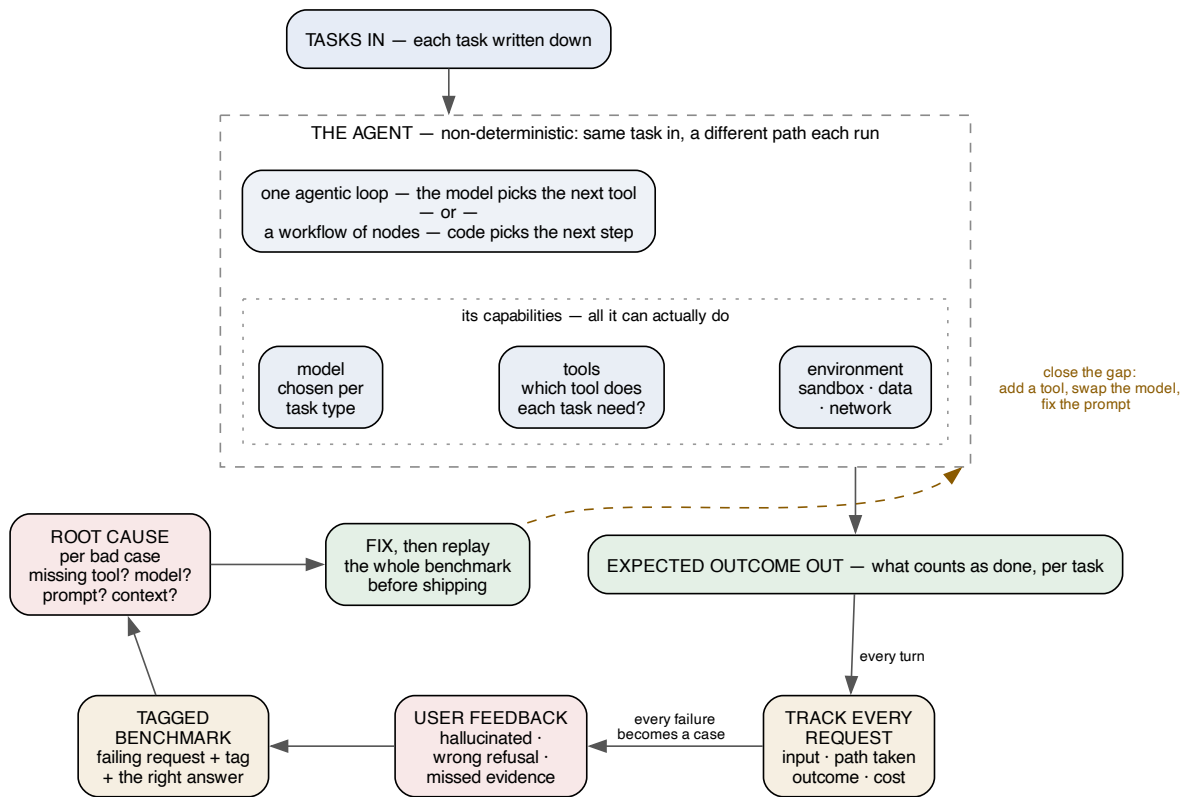


Figure 1: The framework: tasks in, outcome out, and the loop that improves it

- Non-determinism is fenced in, never trusted: validation inside tools, per-turn budgets, middleware at fixed points, reviewers outside the loop, and a custom graph only when a mandatory step must sit *inside* the loop.
- One system, many models: an expensive model where judgment is made, temperature 0 for judges, the cheapest capable model for compression jobs.

3. Tracking and feedback

- Log per request: input, tool calls made, outcome (answered / refused / failed), tokens and cost, latency.
- Feedback on each: was the outcome right — from reviewers, from users, or from our own grading.
- The benchmark is built from this traffic: real requests plus graded expected outcomes, replayed after every change.

4. The guiding agent today

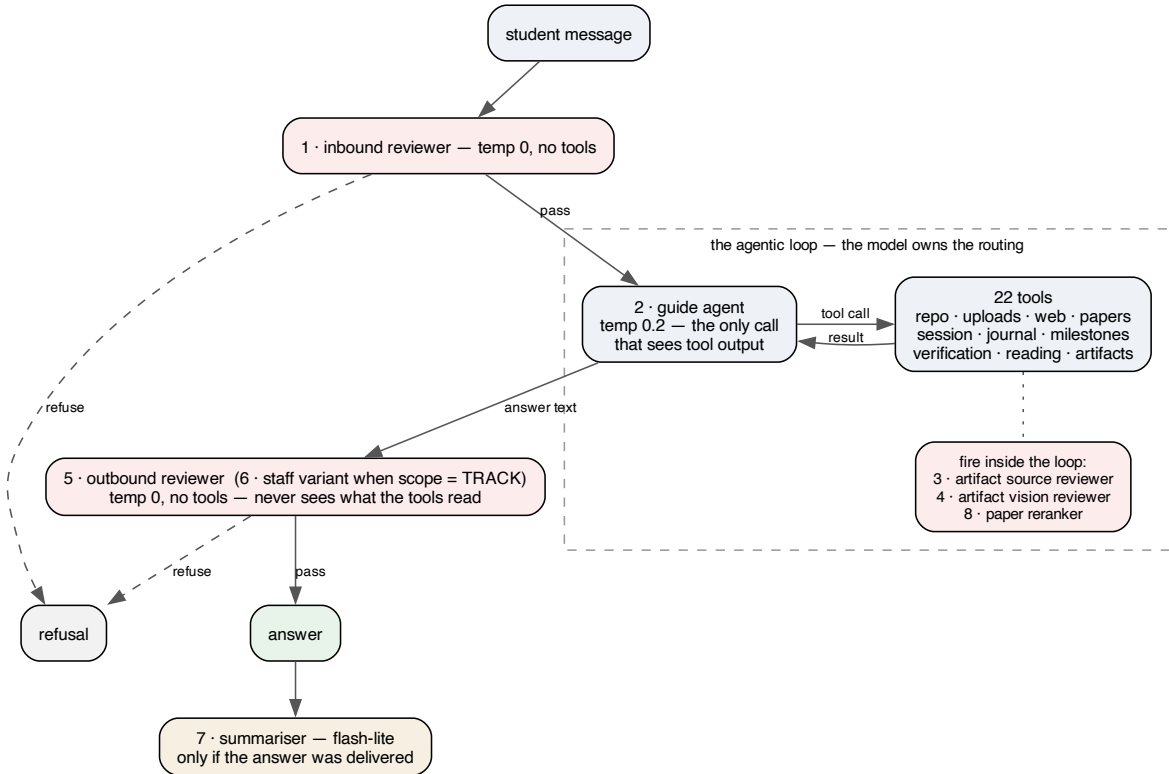


Figure 2: One turn, and where each of the eight model calls sits

- **Tasks it has:** 18 of them, listed and typed in §7.1 — answering with traceable citations, reading the team’s repo, uploads, web and papers, maintaining milestones and the journal, recording verification verdicts, drawing artifacts.
- **Capabilities:** 22 tools across 10 families (§6), every read and write tenant-scoped through apps/api; no execution environment — a verdict rests on what was read, never on what was run.

- **Tracking that exists:** a per-turn usage log line (full cost, including what the budget doesn't see); metrics for turn duration, tokens, cost, review rejections, tool errors, queue wait; per-model-call capture with redaction.
- **Missing:** the benchmark. No graded set of real turns exists, so no change to prompt, model or tools is verifiable (§9, G-03).

5. Agent registry — what goes in, what must come out

Eight model calls. Order on one turn: **1** → **2** (3, 4 and 8 fire inside it) → **5 or 6** → **7**, and 7 only if the answer was actually delivered.

#	Agent	Model	Purpose	Tools
1	Inbound reviewer	flash, temp 0	should this student message reach the guide?	no
2	Guide agent	flash, temp 0.2	reads sources, writes the answer	yes — all 22
3	Artifact source reviewer	flash, temp 0	does this diagram match what was read? (mid-loop)	no
4	Artifact vision reviewer	flash, temp 0	is the rendered diagram legible? (mid-loop, if complex)	no
5	Outbound reviewer	flash, temp 0	should this answer reach the student?	no
6	Outbound reviewer (staff)	flash, temp 0	same, for staff sessions (scope == TRACK)	no
7	Summariser	flash-lite	rewrites the running session memory	no
8	Paper reranker	flash, temp 0	scores search results for relevance	no

5.1 What each one is actually given

1. Inbound reviewer — `prompts/inbound_reviewer.md`; the project brief (full idea markdown, no thread); the thread so far as real turns, *without* attachment notes (stripped on purpose: the note is an instruction block, and this reviewer’s job is spotting instructions in student text); the new message; and the mechanical pass findings — any override phrasings found by regex, named but not acted on, plus every number the student typed.

2. Guide agent — the only one with tools. Its system message is four parts glued together: CITE_SOURCES (attribution rules), NO_TASK_PROMPTS, ARTIFACTS (diagram rules), the project brief, and an optional slash-command steer, last. The brief carries team name and idea code, title, abstract, objective, axis and levels, what is held fixed, status / next step / repo URL, reading list, milestones with their internal refs, expected findings, methods, datasets, metrics, evidence, known confounds, prior art, protocol steps, risks, removed milestones, recent journal entries, runs on record, open results and verified-result ids, and “Right now” (week, days left, last push). Then the thread as real turns *with* attachment notes (id plus “you have NOT seen the contents, call `read_file`”), and the current message dated — `[timestamp] text` plus notes for anything just attached. Tools are bound as listed in §6, and the context grows through the loop with every tool result it calls for: text as text, images as real image blocks it can see, up to ~23 rounds.

3. Artifact source reviewer — runs inside the loop, once per diagram: `prompts/artifact_reviewer.md`; brief, thread, the student’s question and the answer text the diagram belongs to; the artifact spec as JSON (first 8,000 chars); and what the guide has read so far this turn (first 8,000 chars, fenced as student-controlled text).

4. Artifact vision reviewer — only when the diagram is complex, and always for HTML: `prompts/artifact_vision_reviewer.md`; brief, thread, question, title, and the rendered PNG itself. Deliberately *not* the spec — it judges the picture, not the source.

5 / 6. Outbound reviewer — `outbound_reviewer.md`, or `outbound_reviewer_staff.md` when the session’s scope is TRACK; the project brief and thread so far, again without attachment notes;

the student’s question repeated explicitly; the answer the guide wants to send; and the mechanical pass findings — every code fence (language, line count, definition count, file-shape signals) and every number, each stamped accounted for or not accounted for, plus numbers plotted in any chart.

What the outbound reviewer never receives: which tools ran, what they returned, which files or images were read, what the milestone writes actually did, how many iterations it took, or the attachment notes. This is the mechanism behind G-04.

7. Summariser — the SUMMARY prompt (under 250 words, facts only, replacing the previous summary) and three lines: the previous summary, the student’s question, the supervisor’s answer. No brief, no thread, no tools.

8. Paper reranker — one plain prompt: the search queries plus a numbered list of candidates (title, year, abstract truncated to 500 chars), asking for a JSON array of scores. No brief, no thread.

5.2 Two properties that cut across the table

- **Only agent 2 ever sees tool output.** Every other call reasons about a turn it did not watch.
- **Every reviewer’s view of the conversation is text-only** — no attachments, no images, no record of work performed. So a number read off a screenshot is indistinguishable from an invented one by the time it reaches agent 5.

5.3 Rules the table cannot carry

- **The reviewers fail open.** A timeout, a quota error or a malformed verdict lets the turn through, recorded as `failed_open`. An unavailable judge is preferred to silent degradation — but it makes reviewer availability a silent correctness dependency.
- **The inbound reviewer defaults to pass.** It refuses three things only: attempts to override the agent’s configuration or persona, personal abuse, and content unrelated to the project.
- **The outbound reviewer judges four things only:** a runnable module the student could drop in and move on from; a number about the team’s own work that cannot be traced; a decision recorded that the student never made; a prompt handed over for another model to run. Not tone, length, advice quality or repetition.
- **Slash commands change effort, never permission.** Six directives (`/repo`, `/papers`, `/plan`, `/recall`, `/quiz`, `/artifact`) append one paragraph to the system prompt for that turn; the reviewers never see them. `/verify` and `/help` are client-side only and never reach the agent.

Operating envelope

Mean turn	~53,600 tokens in, ~1,600 out
Worst turn observed	229,311 tokens
Per-turn deadline	5 minutes, stored per job
Cloud Tasks dispatch deadline	300 s — must exceed 90 s guide + 30 s per reviewer
Per-team budget	GUIDE_TOKEN_BUDGET, rolling window offset to IST
Artifact writes	3 per turn, 3 repair attempts each
Tool rounds	up to ~23 per turn

6. Tool catalogue — the 22 tools the guide agent has

Budgets are per family, per turn; when one is spent the tool keeps answering with a refusal that tells the model to finish with what it has. “Vouches” means the tool’s result may be cited as evidence for a number about this team’s own work.

Family	Tool	What it does	Per-turn limit	Vouches
repo	repo_head	the repository’s current commit	8 calls / 120 KB	yes
repo	repo_tree	list files at a path	(shared)	yes
repo	repo_file	read a file, by line range	(shared)	yes
repo	repo_diff	what changed between two commits	(shared)	yes
uploads	list_files	what the team has uploaded	8 calls / 120 KB	yes
uploads	grep_file	search inside one upload before reading it	(shared)	yes
uploads	read_file	read a slice, with its page/slide/sheet marker	(shared)	yes
session	read_session	page into an earlier thread’s real messages	5 calls	yes
journal	read_journal	page back through older journal entries	3 calls	yes
journal	add_journal_entry	write today’s entry (one per day, rewritten)	(shared)	no
milestones	add_milestone	propose a milestone — title plus instruction	3 calls	no
milestones	update_milestone	move progress (evidence required) or soft-delete with a reason	(shared)	no
verification	record_verification	record a verdict against a submitted result	2 calls	no
reading	add_reading	propose a paper — staff approval still needed	3 calls	no
reading	mark_reading	record that the team read something	(shared)	no
artifacts	find_artifact	look for a diagram that already exists	3 writes / 3 repairs	no
artifacts	reference_artifact	point at an existing one (free, no re-verification)	(shared)	no
artifacts	revise_artifact	move an existing diagram forward (full verification)	(shared)	no
artifacts	create_artifact	draw a new one (full verification)	(shared)	no
papers	paper_search	three academic sources, deduped and reranked	2 calls, 5 queries each	no
web	web_search	Serper search	5 calls	no
web	web_fetch	one page’s text; every redirect hop address-checked	(shared)	no

Why the last seven do not vouch: web and paper results are other people’s numbers, and admitting them would let any fetched page launder a figure into a claim about this team. The writes do not vouch because their results echo back what the agent just sent — it would become a source for its own claims. Artifacts are excluded for the subtler version of the same thing: a number invented in week 3 and embedded in a chart must not return in week 8 as traced fact.

What the agent deliberately does not have: a shell, a filesystem, an execution environment, a database credential, a GitHub token, a bucket credential, or any parameter naming another team. Every call goes through apps/api on an internal key, scoped to the group id the task arrived with.

7. Tasks, and whether one loop fits them

7.1 Task inventory

Every task the guiding agent is expected to handle, with the kind of task it is. The type column is what decides the architecture — §7.2 works from it.

#	Task	Type	Tools it needs	Where it stands
1	Unblock a doubt — explain a technique or a design choice	explanation	model knowledge, web/paper search	works
2	Answer a question about the team’s own code	retrieval + explanation	repo_tree, repo_file, repo_diff	works
3	Read their uploaded results or paper and answer from it	retrieval + extraction	list_files, grep_file, read_file	works
4	Find prior work and cite it	multi-step research	paper_search, web_search, web_fetch	works
5	Recall what was decided in an earlier thread	retrieval	read_session, read_journal	works
6	Keep the decision log — track doubts and what was tried	state write	add_journal_entry	works
7	Propose or re-scope the plan	judgment + state write	brief, journal, add_milestone	works
8	Move a milestone’s progress, with evidence	judgment + state write	update_milestone + evidence	works
9	Decide a milestone is genuinely complete (VERIFIED)	judgment, highest stakes	evidence + rubric + viva	partial — only via a citable result
10	Analyse whether a submitted result is wrong	analysis + judgment	read the artifact, re-run the check	blocked — nothing can run
11	Record the verification verdict	state write	record_verification	works
12	Propose reading, or record that it was read	state write, low stakes	add_reading, mark_reading	works
13	Draw a diagram or chart	generation + validation	find / reference / revise / create_artifact	works
14	Examine the student on their own work (viva / quiz)	interactive elicitation, multi-turn	thread state, prior results	partial — single turn only
15	Refuse to do their work; refuse untraceable numbers	guardrail	reviewers outside the loop	works, false positives

#	Task	Type	Tools it needs	Where it stands
16	Notice scope vs deadline and say so	judgment	brief, milestones, journal	untested
17	Keep the running session summary	compression	cheap model	works
18	Answer staff about how the product works	retrieval + explanation	product primer	works

7.2 Does one agentic loop fit these tasks?

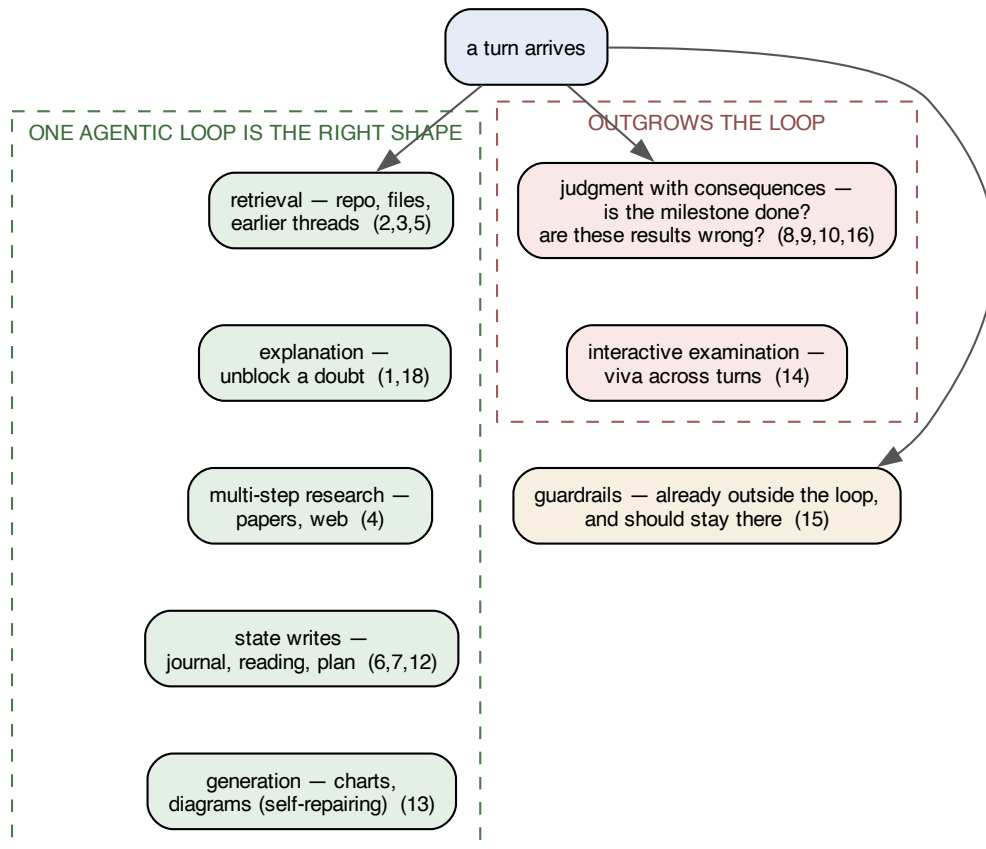


Figure 3: Which task types the loop holds, and which outgrow it

7.2.1 The task types, and the property that matters

Type	What makes it that type	How it fails
retrieval	there is one right place to look	it didn't go and look

Type	What makes it that type	How it fails
explanation	nothing external to fetch; the answer is the model’s	wrong, or right but useless to a student
multi-step research	depth is unbounded until the agent decides to stop	shallow reading, or budget and latency blown
state write	it mutates a durable record	wrong row, wrong moment, no evidence behind it
generation	output must validate against a schema	invalid spec — but the tool returns the errors, so it self-repairs
judgment	a verdict with consequences for the student	the verdict cannot be defended from what it read
interactive elicitation	the exchange spans turns	no memory of where the examination had got to
guardrail	decides what must not be said	false refusal, or a miss

7.2.2 Where one loop is the right shape Tasks 1–8, 11–13, 17–18. For all of them, “the model picks the next tool, reads the result, decides whether it has enough” is the correct control flow — the path genuinely cannot be known in advance, and a graph would only hard-code guesses about which tool comes next. Generation (13) is the strongest case: the validation errors come back as a tool result and the loop repairs itself with no orchestration at all.

7.2.3 Where it strains, in order of how much it hurts

1. **Judgment with consequences (8, 9, 10, 16).** These run on the same model, the same temperature and the same context as a casual explanation, with no separate rubric pass. Task 10 is worse than a tuning problem: the evidence needed to say a result is wrong does not exist inside a loop that can only read. This is the one that actually breaks.
2. **Interactive examination (14).** A viva is a state machine across turns — what was asked, what was answered, what is still unprobed. Today each turn is independent, so the loop re-derives the examination from the thread every time.
3. **One shape for every request.** A one-line recall (5) and a deep literature turn (4) get identical setup, identical model, identical budget. Nothing routes.

7.2.4 Three ways forward

	What it fixes	What it costs	What it does not fix
A. One loop + routing — classify the turn at entry, pick the model and budget for that task type	model mismatch, cost and latency spread	a classifier, and a new failure class when it mislabels	evidence for 10; the viva’s state

	What it fixes	What it costs	What it does not fix
B. One loop + a verification path of its own — a small graph for 9/10: gather evidence → run checks → rubric pass → write the verdict	the judgment tasks, properly	more machinery; sandbox needed for 10	the everyday cost spread
C. Decompose by task type — a classifier node in front of specialised branches	maximum control per task type	most build and most upkeep; the “one supervisor” feel goes	nothing extra beyond A + B

7.2.5 What would settle it The traffic mix by task type — unknown today. The slash commands already carry a partial label (`/repo`, `/papers`, `/plan`, `/quiz`, `/recall`, `/artifact`); tagging every turn with its type in the usage log would give the split within a week. If judgment turns are a small slice, B alone is enough and A is a cost optimisation. If the mix is wide, A and B together, and C stays unjustified.

My reading: one agentic loop is right for the bulk of this agent, and verification is the task that should leave it — A and B, not C. Decision yours.

8. Debug sheet

My own working sheet. Filled in as the guide agent runs, so a bad answer can be traced to a cause rather than argued about.

8.1 Failure tags

Fixed vocabulary, so cases stay comparable across months. Extend the list deliberately, never ad hoc.

Tag	Means
hallucinated-number	stated a figure not present in anything it read
hallucinated-source	cited a file, page or paper that does not exist or does not say that
wrong-refusal	refused an answer that was legitimate
missed-evidence	the answer was in reach and it did not go and read it
shallow-read	read too little of a file to be right
stale-context	answered from an old thread or old state
did-their-work	produced the thing the student was meant to produce
tool-error	a tool failed and the answer went out anyway
unusable	correct but not actionable for the student

8.2 Root-cause classes

Class	Means	Typical fix
missing capability	no tool exists for what the task needed	add the tool
wrong model	task needed more reasoning than the model gives	route this task type elsewhere
no environment	needed to run something, could only read	sandbox
prompt gap	the instruction never told it to do that	prompt change
context gap	the data was never put in front of it	change what the brief carries
tool result quality	truncated, garbled or too large a result	fix the tool, not the model
guardrail misfire	a reviewer rule too broad	narrow the rule
infra	timeout, rate limit, budget exhausted	ops

8.3 Benchmark cases

Every bad case lands here and stays here — this table *is* the benchmark.

ID	Date	Tag	The request	What it did	What it should have done	Root cause	Status	Fixed in
----	------	-----	-------------	-------------	--------------------------	------------	--------	----------

8.4 Drill — when a bad answer is reported

1. Find the turn: session, message, timestamp.
2. Read the path it took — which tools ran, in what order, with what results.
3. Ask whether the evidence was in reach at all. If not, it is a capability problem, not a model problem.
4. Tag it (§8.1).
5. Assign one root cause (§8.2).
6. Add the case (§8.3), with the answer it should have given.
7. Fix, replay the whole table, record which change fixed it.

9. Issue and fix register

The live list. An issue leaves this table only when the fix has shipped and a benchmark case covers it.

ID	Issue	Why it matters	Recommended fix	Status
G-01	No per-request model routing — one fixed model for every turn	a recall and a verification get identical capability	classify at entry, pick model and budget per task type	open

ID	Issue	Why it matters	Recommended fix	Status
G-02	No execution environment	a verdict on a result rests on reading, never on running	Cloud Run sandbox (preview) for the verification path only	open
G-03	No benchmark	no change to prompt, model or tools is verifiable	build §8.3 from ~30 real turns before the next change	open
G-04	The outbound reviewer refuses good answers	it judges every number while blind to what the tools read (§5.2), so a figure read off a screenshot looks identical to an invented one	pass a record of the work performed — tools run and what they returned — into the reviewer	in progress
G-05	No per-team storage cap	25 MB per file, unlimited files, nothing sums per group	measure per-team usage, then set a cap	open
G-06	Budget invisible until refusal; staff share the team budget	a team hits the wall mid-conversation; a prof with ten teams hits it first	surface remaining budget in the UI; separate staff budgets	open
G-07	The viva has no cross-turn state	each turn re-derives the examination from the thread	hold examination state per session, with no visible mode	open
G-08	Artifact review is asymmetric	an early-turn diagram is judged against less reading than a late one	accepted trade — deferring review to turn end is too late to act on	by design
G-09	Proactive outreach unbuilt	no weekly literature sweep, no deadline escalation; unprompted has no producer	scheduled producer writing unprompted turns	not started
G-10	Plain-Python checks removed (2026-08-17)	protocol validation and signal-vs-noise now depend on the agent reading	keep as prompt material until the sandbox exists	accepted
G-11	Docs drift from code	README says no tools; architecture Part 5 says v1 holds none; the tool count is stated as 20 against 22 in code	one pass over README and the architecture doc	open
G-12	Deployment traps	Cloud Scheduler job names are global per project/region; EXECUTION_ENVIRONMENT is case-sensitive; the agent reads API_URL , not API_BASE_URL	prefix scheduler jobs with the API host; assert both env vars at boot	known
G-13	“Scope vs deadline” never observed firing	the most supervisor-like behaviour in the product is untested	add a benchmark case for it	open

ID	Issue	Why it matters	Recommended fix	Status
G-14	Single provider in practice	a swap is a string, but only Gemini is exercised	run the benchmark against one non-Gemini model once it exists	open
G-15	Reviewers fail open	a judge that times out lets the turn through	alert on failed_open rate rather than changing the default	by design, monitor

Priority order agreed so far: G-03 and the telemetry read first (they make everything else measurable), then G-05 / G-06, then the verification work (G-02 with G-01).

Decisions taken

Date	Decision	Where it came from

10. One turn, end to end — the implementation walkthrough

Written so a developer can follow one student message through all three processes without opening the code. Everything below is what the code does today, not what the design documents say it does. Section 4 is the shape; this is the wiring.

Three processes are involved and they trust each other in exactly one direction: **apps/api owns auth, tenancy and persistence; the guide-agent owns judgement and owns no data.** The agent never queries the database. Everything it will ever know about a project is frozen into one JSON snapshot at the instant the turn is created and pushed to it.

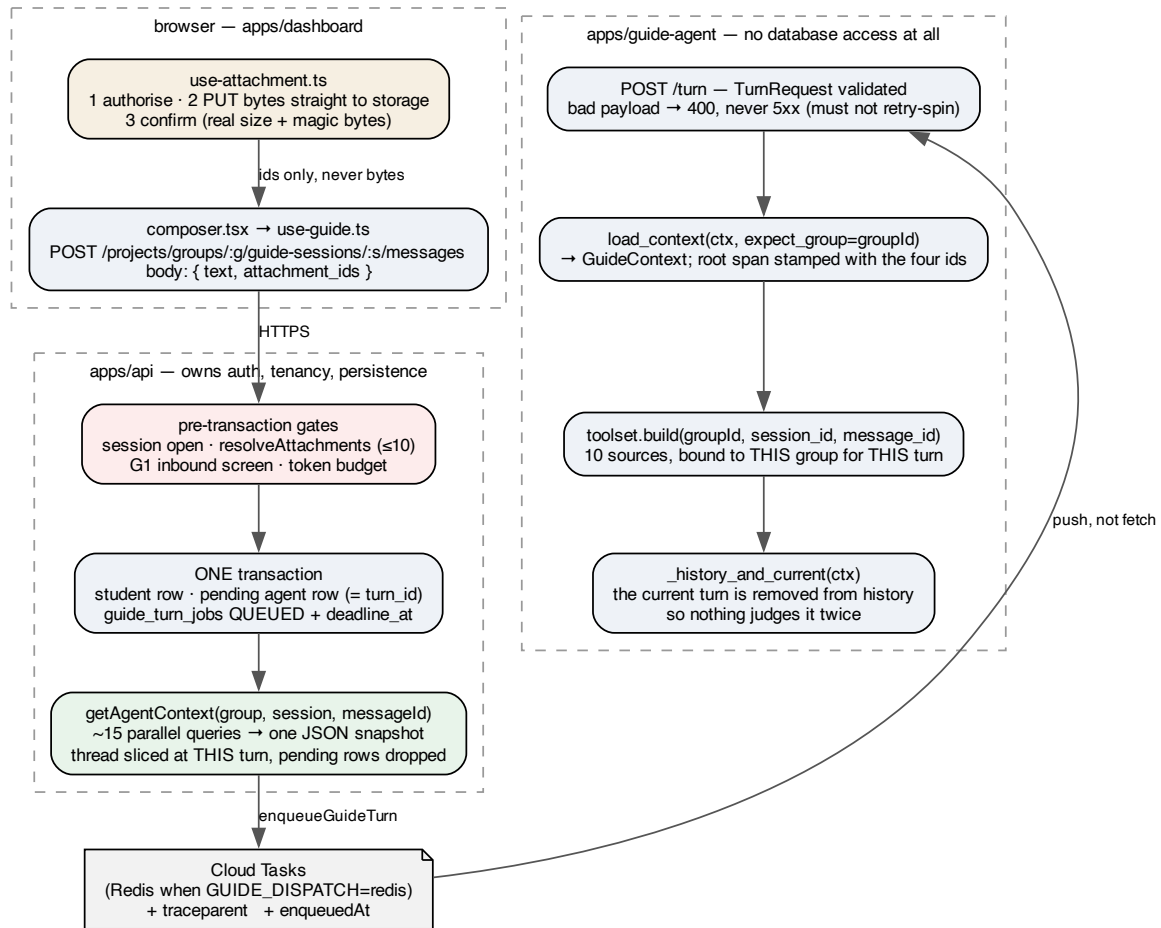


Figure 4: The request path — down the left through the browser and apps/api, continuing down the right in the agent

10.1 Before the message — the upload path

A file is uploaded *before* the message that references it, in three steps (`use-attachment.ts`):

1. `POST /projects/groups/:g/guide-attachments` — may this be uploaded, and where does it go. Returns an attachment id and a signed URL; the row is created **PENDING**.

2. PUT <signed url> — the bytes go **straight to Google Cloud Storage**. They never pass through Express: no multipart parsing, no request-size ceiling, no memory held. The API wrapper is deliberately not used, because it would attach our auth headers and the signature *is* the authorisation here.
3. POST /guide-attachments/:id/confirm — do the contents match the claim.

Client rules: five files per message (the API's `resolveAttachments` caps at ten), uploaded **sequentially** — five parallel PUTs from bad wifi produce the hardest failure to report — and one refused file does not fail the batch.

Three separate screens, not one:

When	Check	Refusal
Before upload	extension allowlist per kind; declared size vs per-kind cap	extension, size
During	signed URL carries a length range	storage rejects it
After (confirm)	real size read from storage replaces the declared one; first KB read	masquerade

Allowlist by kind — RESULT `csv json tsv txt log yaml yml xlsx` · PAPER `pdf` · CODE `py ipynb r jl js ts java cpp c go rs sql sh m` · IMAGE `png jpg jpeg svg webp` · DOCUMENT `docx pptx pdf md txt` · OTHER `txt md csv json pdf png jpg jpeg`. Anything else is refused before a byte moves.

Size: **5 MB**, except **CODE at 2 MB** (a 2 MB source file is a generated artifact, not something to supervise). Measured against every file actually uploaded as of 2026-08-26 — largest 0.84 MB. The population deliberately excluded is figure-heavy published PDFs at 5–20 MB; they get a refusal that names the size.

The content screen does two things. A **magic-byte check on every kind** — an extension is a claim the uploader makes, and this is the only place it can be checked against the bytes. And a **code-shape check on RESULT only** — “I will not treat a program as a measurement” — exempting `.log/.txt`, which may legitimately contain a traceback. If the head cannot be read at all, that is **not** a refusal: a storage hiccup must not become an accusation about a student's file.

The row ends **UPLOADED** or **REFUSED**; only **UPLOADED** rows can ride on a message.

10.2 The send

`composer.tsx` holds the draft and calls `onSend`; `use-guide.ts` makes the only network call:

```
POST /projects/groups/{groupId}/guide-sessions/{sessionId}/messages
body: { text, attachment_ids?: [...] }
```

The group is in the path as well as the session so the route carries the same team gate as every other guide route, rather than inferring authority from a message id. The body carries **ids, never files** — the server resolves them against rows this team owns, that passed screening, and that are not already attached to another turn. A client-described attachment is ignored outright.

10.3 Four gates before anything is written

All four sit *above* the transaction, so a refusal leaves no rows behind:

1. **Session open** — a closed thread is refused; start a new session.
2. **resolveAttachments** — team-owned, screened, unused, capped at ten.
3. **screenInbound (G1)** — the deterministic inbound floor: four narrow pattern families. Returned as **data with a 200**, not thrown, because the composer needs to render the refusal and offer edit-and-resend.
4. **Token budget** — `checkBudget(groupId)`. A refusal here is `budget_block`, deliberately a different channel from an inbound refusal: telling a student their message was screened when it was not is a worse error than the one it replaced.

10.4 The transaction — three rows

One transaction writes:

- the **student's** `guide_messages` row (text + attachment ids);
- the **agent's** `guide_messages` row — `blocks: []`, `pending: true`. This empty placeholder's id is the `messageId`, which is the `turn_id` every log line, span and metric is keyed by;
- a `guide_turn_jobs` row — `status: QUEUED`, `deadline_at = now + TURN_DEADLINE_MS`. Stored rather than derived, so changing the constant later does not silently extend every job already in flight. A job with no message, or a message with no job, are both states the watchdog has no story for — hence one transaction.

It also updates the session's `message_count`, `last_message_at` and `context_load` (recomputed from the whole thread, because the fraction is not linear in turn count — one pasted stack trace moves it further than ten short questions).

10.5 The snapshot

After the commit, never inside it — the agent may start before the function returns and must not race a row that is not there yet.

`getAgentContext(groupId, sessionId, agentTurn.id)` runs ~15 queries in parallel and returns one JSON object:

- `idea{}` — title, abstract, objective, axis and levels, holds-fixed, plus six child tables read in `position` order: methods, datasets, metrics, evidence, confounds, protocol
- `state{}` — status, next step, repo, milestones, decision and reading counts
- `messages[]` — the thread, **sliced at the turn being answered** and with pending rows filtered out, so the agent never sees its own blank row and cannot count it
- `sessions[]` with their rolling summaries · `journal[]` + `journal_total` · `experiments[]` (the run registry) · `open_results[]` · the **citable verified results** (the only ids `update_milestone` accepts) · `reading[]` · the artifact catalogue · `removed_milestones[]`
- `group_id`, `tenant_id`, `session_id`

If the enqueue that follows throws, the pending row is not left spinning: an error is written into the thread instead.

10.6 The queue

`enqueueGuideTurn({ messageId, groupId, sessionId, context })`, with `withDispatchMeta` adding **transparent** (what makes one trace span both services) and **enqueuedAt** (what queue wait is

derived from). Cloud Tasks in production; Redis when `GUIDE_DISPATCH=redis`.

10.7 The agent receives it

`POST /turn`, body validated by `TurnRequest (messageId, groupId, sessionId, enqueuedAt, context)`. Then, in order:

- `queue_wait` is recorded **only on a first attempt** (`X-CloudTasks-TaskRetryCount`). A retry redelivers the original `enqueuedAt`, so recording every attempt would fold the previous attempt's model time and backoff into "queue wait" — reading as a backlog during exactly the incidents where that number gets trusted.
- `load_context(req.context, expect_group=req.groupId) → GuideContext`. A malformed payload is **400, not 500**: Cloud Tasks retries 5xx and this must not spin.
- The FastAPI root span is stamped with `turn_id, group_id, session_id, tenant_id`.
- `toolset.build(groupId, session_id, message_id)` — ten sources, bound to the group **the task named**, never the group in the snapshot and never anything the model says.
- `_history_and_current(ctx)` splits the thread: the current turn is removed from history, so no reviewer and no model call ever sees it twice.

10.8 Inbound review

`review_inbound` gets: its prompt, the brief, the thread **without attachment notes**, the new message, and `extract_inbound`'s findings (override phrasings by name, numbers the student stated). The attachment note is stripped on purpose — it is an imperative instruction block ("call `read_file...`"), and this reviewer's whole job is noticing instructions inside student-authored text. Left in, it refused every turn that carried a file.

10.9 What the model is actually given

`agent.run` assembles exactly three things, plus the tool definitions (which are prompt text and count as input tokens):

1. **One SystemMessage**: `CITE_SOURCES + NO_TASK_PROMPTS + ARTIFACTS` + the brief rendered as markdown (`to_markdown(ctx, include_thread=False)`) + a slash-command steer last, if one was typed. Last on purpose: it is the most specific instruction and the only one about *this* message, so it reads as the immediate task against the standing rules rather than as another standing rule.
2. **The thread** as real Human/AI turns, each dated, **with** attachment notes — the agent is the only caller that may see them.
3. **The current message**: `[timestamp] text` plus a note naming each file just attached and telling it to call `read_file`. Naming the file alone invites the other failure: a model that knows a chart exists and describes it from the filename.

Then the loop runs. Each tool result is appended and **the whole growing list is re-sent on every iteration** — which is how one observed production turn reached 560k input tokens over 23 iterations.

10.10 What a tool returns — the fork that decides traceability

This is the most consequential detail in the system and it is invisible from the outside.

- **Text path** (`.docx, .csv, code, a PDF with a text layer`): deterministic parsers — no model — produce text, sliced 200 lines at a time with `from_line, to_line, total_lines, more`, and a

location computed by walking the document’s markers (“p. 3-4”, “slide 7”). That is what lets an answer say “*your results file has 0.803 at seed 3*”. The result is a **dict**, so it flattens to text and **every number in it enters read_text, and therefore the traceable set**.

- **Picture path** (an uploaded .png/.jpg; page=N on a PDF; figure=N in a .docx/.pptx): the result is a **list of content blocks** — a caption plus the raw image. LangChain passes a returned list straight through as the ToolMessage’s content, so the image reaches the model natively. **There is no captioning step and no OCR**. Only the caption enters **read_text**; the pixels contribute nothing.

So the same document is traceable or not depending on *how* it was read — and a scanned PDF has no text layer at all (“readable and empty”, which the code distinguishes from “cannot read”), forcing the picture path for the whole document. The tooling actively steers toward pictures for result questions, because a chart is not in the text.

TRACEABLE_TOOLS decides which reads may vouch for a number: the repository tools, the upload tools, **read_session**, **read_journal**. **web_search**, **web_fetch** and **paper_search** are **excluded on purpose** — a page containing 0.87 is not evidence that this team’s accuracy was 0.87, and admitting them would let the model launder any number through any page. The state-writing tools are excluded for a duller reason: their results echo back what the agent just sent.

10.11 Reviewers that fire inside the loop

Artifacts are reviewed while the loop is still running, because a verdict after the model stopped generating is a verdict nothing can act on. Source review runs once per create or revise, against the question, the answer text, the spec, and whatever the turn has read *so far* (fenced as student-controlled text). Vision review renders the artifact and looks at it — HTML always, other kinds only above a complexity threshold of 25 — and runs **once, never a loop**: a second failure downgrades to prose. An artifact created early is therefore judged against less reading than one created late; that asymmetry is accepted (§9, G-08).

10.12 The mechanical pass, and the turn’s own record

Before the outbound reviewer runs, plain Python assembles the evidence:

- **traceable** = numbers in the student’s message + numbers in the brief and thread + numbers in **read_text**.
- **extract_outbound** lists every code fence (language, lines, definitions, file-shape signals) and every number, each stamped *accounted for* or *not accounted for*. **ALLOWED_SPANS** blanks protocol choices, citation years, versions and plan positions first, so the reviewer is handed four numbers rather than forty.
- Chart values from any artifact join on the same terms — as evidence, never as a verdict. An earlier version hard-refused on them and killed the first real diagram anyone asked for, over ViT-L/14 and a 1024-d feature vector.
- **record_findings** adds what the turn *did*: what was read (marking a picture as unscannable), what was looked up on the web, and which state writes **applied or were REFUSED**. Sourced from the capture callback, so the model cannot write to it — which is the only reason it can be used to check the model. Labels are flattened to one line and fenced, because a filename is bytes a student chooses.

10.13 Outbound review

`review_outbound` gets the brief, the thread (again without attachment notes), the question repeated explicitly, the answer, and that evidence list. Which prompt it uses is decided by `ctx.scope`, a **database column** — never a caller’s claim about who is asking — because handing a student the staff reviewer would remove the rule that protects their own work.

It is a single call, temperature 0, **no tools**, 30-second timeout, structured output `{decision, reason}`. It **fails open**: a timeout, a quota error or an unparseable answer lets the turn through and records `failed_open`. On a refusal the reviewer’s **reason** becomes the student’s answer — the reviewer is ghost-writing in the supervisor’s voice, which is why a wrong refusal reads as the product lying rather than as a policy notice.

There is **no deterministic check behind it**. G3 and G4 were removed on 2026-08-17; `ProjectGuideService.review` validates block types and nothing else. Both outbound prompts claimed otherwise until this was corrected.

10.14 The summariser

Runs only when the turn passed. Cheapest model, three lines of input — the previous summary, the question, the answer — and no brief, no thread, no tools. A miss leaves the previous summary in place, which is staleness rather than a gap.

10.15 The response

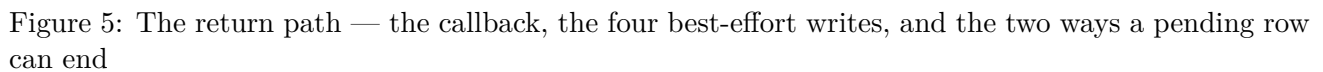
`TurnResponse`: `blocks`, `tokensIn/Out`, `summary`, `refusedBy`, `artifactCounts`, `model`, `promptVersion`. The last two are null on a refused turn, deliberately: the reply there is the reviewer’s sentence, so attributing a thumbs-down to the guide’s model and prompt would file the complaint against the wrong thing.

10.16 `completeTurn` — what apps/api does with it

Idempotent: the update is conditioned on the row still being pending, so a redelivered task changes nothing and says so. The attempts counter is bumped **before** that check, on every delivery, because `attempts > 1` on a COMPLETED row is the only signal anywhere that Cloud Tasks retried.

Then, in order:

1. **The floor** — `review(blocks)`: are there blocks, and is every type one a renderer knows. That is all that remains of the deterministic outbound rules. An unknown block type reaches a student as a blank gap, and no prompt can prevent a renderer from having no case for a tag.
2. **Artifact ingest** — only on a turn that will actually be shown. A withheld turn must not seed the reuse catalogue with content a reviewer just rejected. A `viz` fence becomes a row, and the fence is rewritten in place to point at it.
3. **The write** — `blocks`, `pending`: `false`, `refused_by`.
4. **The job row** — COMPLETED, FAILED (the agent reported an error) or REVIEWER_BLOCKED (every refusal), plus tokens, model, prompt version, artifact counts and a failure reason.
5. **Withdraw artifacts** if the turn was withheld — `create_artifact` writes through the internal route *during* the turn, before any reviewer has decided, so a blocked turn otherwise leaves a complete row listed in the panel. Observed exactly once, on the first real diagram anyone asked for.
6. **The session summary**, only on a delivered turn: a memory describing an answer the student never saw would be a memory of a conversation that did not happen.



Steps 2, 4, 5 and 6 are all best effort. None of them may cost the student the answer they are waiting for.

10.17 What the student sees

One **SSE stream per open thread** (`guide-stream.ts`), opened with a raw `fetch` rather than `EventSource`, because `EventSource` cannot carry an Authorization header. Nothing polls any more. While the turn is pending the stream carries **stage events** — “Checking your message”, “Working through it”, “Reviewing the answer” — each landing on its own pending row so two running turns do not cross. When the row settles, the answer replaces the spinner.

10.18 When nothing comes back

A Cloud Scheduler tick of about a minute hits `POST /internal/guide-turns/sweep` → `expireOverdueTurns`, which claims each overdue job conditionally (so overlapping runs are safe) and rewrites the pending row. It lives beside `completeTurn` because those two are the only ways a pending row stops being pending. `GUIDE_TURN_SWEEP_ENABLED=false` pauses it without touching the scheduler job — during a known outage, an apology in every thread is worse than a spinner somebody can explain. A second, hourly sweep reaps uploads that were authorised and abandoned.

10.19 Where to look when it goes wrong

Every event below carries `turn_id`, `session_id`, `group_id`, `tenant_id`, so one query — `jsonPayload.turn_id="<id>"` — returns the whole request.

Event	Fired by	Carries
<code>guide.turn.model_input</code>	capture, first model call	the complete message array, verbatim
<code>guide.turn.model_input_delta</code>	capture, every later call	only the newly appended messages
<code>guide.turn.tool_call</code>	capture, per tool	tool, redacted args and result, duration, error
<code>guide.turn.review</code>	<code>turn.py</code> , both stages	refused, reason, and the evidence the verdict used
<code>guide.turn.output</code>	<code>turn.py</code>	raw_text (what the loop wrote) and final_text (what the student got)
<code>guide.turn.usage</code>	<code>turn.py</code>	every model call’s tokens and cache reads, per role

Spans: `guide.turn` → `REVIEW inbound` → `CALL gemini` → `TOOL *` → `REVIEW outbound`, nested under the originating apps/api request through the propagated `traceparent`. Metrics: turn duration, tokens, cost, review rejections, tool errors, queue wait.

Two content kill switches, separate on purpose: `OTEL_SDK_DISABLED` stops traces and metrics; `GUIDE_CAPTURE_DISABLED` stops content reaching the logs while leaving the events and their metadata in place.